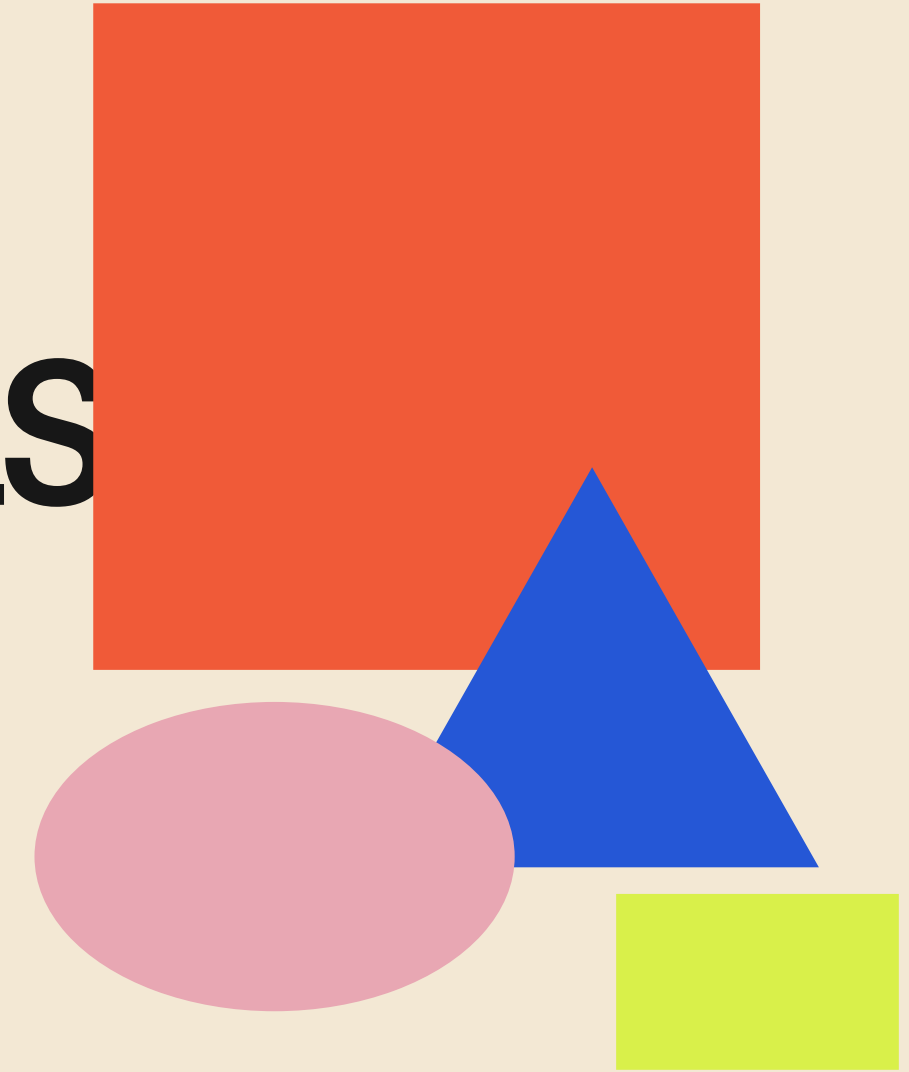
DATA SYSTEMS ADMINISTRATOR · ENTRY LEVEL

LINUX FUNDAMENTALS



Files, folders, drives and the command line —
one command at a time.

No experience needed. Bring curiosity.

COURSE MAP

FIVE STOPS ON THE TOUR

05

units · from zero to job-ready

01 **Linux Fundamentals**
What Linux is, and why the terminal matters

02 **Understanding File Types**
.exe .zip .iso — and what an extension really means

03 **Create, Modify & Move Files**
touch, mkdir, mv, cp, rm — plus file properties

04 **Drives, Media & Paths**
One tree, USB sticks, and finding your way

05 **Commands: bash · CMD · PowerShell**
Three dialects, one idea — plus the power of pipes

UNIT 01

LINUX FUNDAMENTALS

Meet the operating system that quietly runs the internet.

01

WHAT IT IS · WHERE IT RUNS · YOUR FIRST COMMANDS

THE PENGUIN UNDER THE HOOD

Linux is a free, open-source operating system — the basic software layer that makes a computer usable. Anyone can read its code, improve it, and share it.

Instead of clicking icons, administrators talk to Linux by typing short commands into a terminal, called the **shell**.

100%

of the world's 500 fastest supercomputers run Linux

3B+

Android phones and tablets run on a Linux kernel

$0

what it costs to download, use and share — free as in freedom

EVERY COMMAND IS A SENTENCE

```
student@pc:~$ ls -l /home
```

ls — the **verb:** what to do (list files)

-l — the **option:** how to do it (long, detailed list)

/home — the **argument:** where to do it

STUCK? HELP IS BUILT IN

man ls — the full manual page

ls --help — a quick summary

Tab key — finishes file names for you

Typing the Windows command dir in bash sometimes works (it's an alias on some systems) — but learn ls. Trust the man page over a random forum post.

WHY NOT JUST CLICK?

500

miles away sits a server with no screen, no mouse, no desktop.

ssh user@server — and you're in. The command line is your hands.

01 **Servers are headless** — most have no graphical desktop at all

02 **Scripts repeat perfectly** — three lines beat two hundred clicks

03 **Remote control** — fix any machine, from anywhere, over ssh

04 **It's the job** — sysadmin, cloud and security roles expect CLI skills

Automation tools like Ansible and Docker are command-line first — this is a career skill, not a history lesson.

UNIT 02

UNDERSTANDING FILE TYPES

.exe .zip .iso .sh .yaml .log

What's actually inside those letters after the dot?

EXTENSIONS VS. CONTENT · PROGRAMS · ARCHIVES · CONFIGS

02

THE EXTENSION IS JUST A NAME TAG

notes.txt
an ordinary text file



notes.jpg
renamed — but nothing inside changed



$ file notes.jpg
→ still ASCII text

Linux doesn't trust the name. It opens the file and reads the first bytes — the magic bytes — to learn what it really is.

Renaming never converts anything. A .txt called .jpg is still text, and a real photo opened in a text editor shows binary gibberish.

THINK

If Linux doesn't need extensions, why do we still use them? (For humans — and some apps.)

Windows leans on extensions to decide what opens a file. Linux asks the file itself.

RUN IT · PACK IT · MOUNT IT

RUN

.exe / .sh

Programs. A .sh script is plain text — give it permission with chmod +x run.sh, then start it with ./run.sh

PACK

.zip / .tar.gz

Packed suitcases. tar -xvf bundle.tar.gz unpacks one. Fun fact: plain .tar bundles but doesn't compress.

MOUNT

.iso

A whole disc in one file. Mount it to look inside, or write it to a USB stick to install an operating system.

Scripts don't run themselves like .exe files — the execute permission and the ./ prefix are doing the work.

CONFIGS, LOGS & HIDDEN DOTS

01	.yaml / .yml	Settings humans can read — **indentation is the grammar: spaces, never tabs.** You'll meet it in Docker, Ansible and cloud configs.
02	.conf / .log	**System settings live in /etc; the system's diary lives in /var/log.** Both are plain text — open them and read.
03	.deb / .rpm	**Installer packages** — the Linux answer to setup.exe, handled by the system's package manager.
04	.bashrc (dotfiles)	**A leading dot means hidden** — plain ls skips them; ls -a reveals your personal settings files.

ASK — why would a system hide its own settings files by default?

UNIT 03

CREATE, MODIFY & MOVE FILES

Touch it, name it, move it — and (carefully) delete it.

03

TOUCH · MKDIR · MV · CP · RM · PERMISSIONS

MAKE SOMETHING FROM NOTHING

touch newfile.txt	an empty file appears (also refreshes a file's timestamp)
mkdir -p labs/week1	builds nested folders in one shot — the -p does the parents too
nano notes.txt	a friendly text editor, right inside the terminal
echo "hi" > a.txt	sends text into a file — no editor needed

>

overwrites — the old content is gone

>>

appends — adds to the end, safely

One wrong > can erase a file you've worked on for hours. Pause before you press Enter.

mv: ONE TOOL, TWO JOBS

JOB 1 · RENAME

mv old.txt new.txt

same place, new name — instant

JOB 2 · RELOCATE

mv notes.txt /tmp/

new home, same name

There is **no separate "rename" command** — mv does both.

mv **overwrites silently**. Use **mv -i** and it will ask first.

Moving inside one drive is instant; moving **across drives = copy + delete**, so it takes time.

THINK

Why is renaming instant, but moving to a USB stick slow?

rm IS FOREVER

NO

recycle bin. No undo. No trash folder.

GUI deletions go to trash — CLI deletions just go.

cp -r project/ backup/

copy before you risk — -r copies whole folders

rm -ri olddir/

interactive mode — it asks about every single file

rmdir emptydir/

only works on empty folders — safe by design

rm -rf

the chainsaw — force + recursive. Never aim it at / and never mix it carelessly with sudo.

IF IN DOUBT — BACK UP FIRST, DELETE SECOND

READ A FILE'S ID CARD

```
-rwxr-xr-- 1 student student 4.2K Jul 21 10:00 run.sh
```

permissions	owner · group	size	last modified	name
who may read, write, execute	whose file it is	on disk	stat shows all timestamps	just a label

CHANGING PERMISSIONS — chmod

`chmod 755 run.sh` — owner can do everything, others read & run

`chmod u+x run.sh` — same idea in letters: give the owner execute

`chmod 777 on everything` = leaving every door in the building unlocked

Deep dive:

stat run.sh reveals the full metadata — access, modify and change times, and the inode: the file's true identity, even after a rename.

UNIT 04

DRIVES, MEDIA & PATHS



One tree. No drive letters. Everything hangs under /

04

THE SINGLE TREE · THE MAP OF / · USB MOUNTING · FINDING YOUR WAY

NO C: DRIVE. ONE TREE.

THE WINDOWS WAY — LETTERS

C: — Windows itself

D: — maybe your photos

E: — today's USB stick

Every drive is its own little island with a letter.

THE LINUX WAY — ONE TREE

/ — the single root of everything

/home/you — your files, always the same address

/media/usb — a plugged-in stick becomes a branch

Drives attach to the tree as folders. No letters needed.

lsblk — show every drive · **df -h** — show free space, human-readable

THE MAP OF /

/home	your files and your desktop life
/etc	settings for the whole system
/var	logs — the system's diary. Troubleshooting? Start in /var/log
/usr	programs and shared tools
/tmp	scratch space — wiped on reboot
/bin /sbin	the basic commands themselves live here

Watch the trap: /root is the admin's home folder — not the same as /, the root of everything.

PLUG IN · MOUNT · EJECT

1	lsblk	spot the new device — there, sdb1
2	sudo mount /dev/sdb1 /mnt	attach the stick to a folder
3	cp report.pdf /mnt/	use it like any other folder
4	sudo umount /mnt	always eject before pulling the stick!

WHY EJECT?

The system caches writes. Pull early and the last files may never reach the stick — corrupted data.

ADDRESSES — ABSOLUTE VS. RELATIVE

/etc/hosts — absolute: starts at the root /

../docs — relative: starts where you're standing (.. = up one)

~ means your home · pwd asks "where am I?" · Tab completes names

UNIT 05

BASH · CMD · POWERSHELL

Three dialects of one idea — tell the computer exactly what to do.

05

THE TRANSLATION TABLE · ALIASES · PIPES

SAME TASK, THREE DIALECTS

TASK	BASH (LINUX)	CMD (WINDOWS)	POWERSHELL
List files	**ls**	dir	Get-ChildItem
Where am I?	**pwd**	cd	Get-Location
Copy	**cp**	copy	Copy-Item
Move / rename	**mv**	move	Move-Item
Delete	**rm**	del	Remove-Item
Read a file	**cat**	type	Get-Content
Built-in help	**man**	help	Get-Help

PowerShell even answers to ls and cd — they're aliases. But under the hood it pipes objects, not plain text.

READ IT LIKE A SENTENCE

cat /var/log/syslog | grep error | wc -l



cat /var/log/syslog — "show me the whole system log"

grep error — "keep only the lines containing error"

wc -l — "count what's left"

42

errors in today's log —
found in two seconds

Small tools, one job each, chained with the pipe | — that's the Unix way. Build pipelines one stage at a time.

YOUR TURN AT THE KEYBOARD

01	Open a terminal and run ls -l in your home folder — read every column
02	Rename a .txt to .jpg and prove with the file command that nothing changed
03	Build labs/week1 with one mkdir command, then fill it with touch and nano
04	Plug in a USB stick, find it with lsblk, then mount, copy, and eject it safely
05	Count the errors in a log with one pipeline: cat /var/log/syslog | grep error | wc -l

Stuck? man and --help are always there. Breaking things in a lab is how admins are made.

OPEN A TERMINAL · TYPE · BREAK THINGS · FIX THEM